

Dynamic Task Parallel Algorithms

Chapter 26

Introduction to Algorithms

Cormen, Leiserson, et al.

breakBundleOfSticks Algorithm

```
for( int stick = 0; stick < bundle.length; stick++ )  
    break a stick from bundle[ stick ]
```

breakBundleOfSticks Algorithm

```
void breakBundleOfSticks( int bundle[], int first, int last )  
{  
    base case  
    general case  
        mid = find middle index  
        breakBundleOfSticks( arr, first, mid)  
        breakBundleOfSticks( arr, mid+1, last )  
}
```

breakBundleOfSticks Algorithm

```
void breakBundleOfSticks( int bundle[], int first, int last )
{
    if ( first == last)
        break a stick bundle[ first ];
    else {
        mid = find index of middle stick
        breakBundleOfSticks ( bundle, first, mid )
        breakBundleOfSticks ( bundle, mid+1, last )
    }
```

Practice question

- How fast can the minimum number be found among N numbers?
- Min finding parallel algorithm?

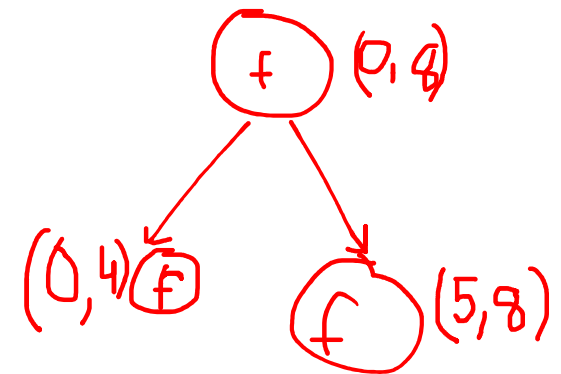
findMin Algorithm

```
int findMin( int arr[], int first, int last )
{
    if ( first == last)
        return arr[first];
    else {
        mid = find middle index
        int lMin = findMin( arr, first, mid)
        int rMin = findMin( arr, mid+1, last )

        return min( lMin, rMin)
    }
}
```

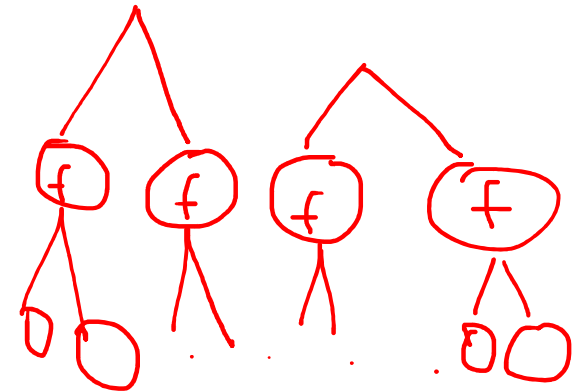
findMin Algorithm

```
int findMin( int arr[], int first, int last )  
{  
    if ( first == last )  
        return arr[first];  
    else {  
        mid = find middle index  
        int lMin = findMin( arr, first, mid )  
        int rMin = findMin( arr, mid+1, last )  
        return min( lMin, rMin )  
    }  
}
```



findMin Algorithm

```
int findMin( int arr[], int first, int last )
{
    if ( first == last)
        return arr[first];
    else {
        mid = find middle index
        int lMin = findMin( arr, first, mid)
        int rMin = findMin( arr, mid+1, last )
        return min( lMin, rMin)
    }
}
```



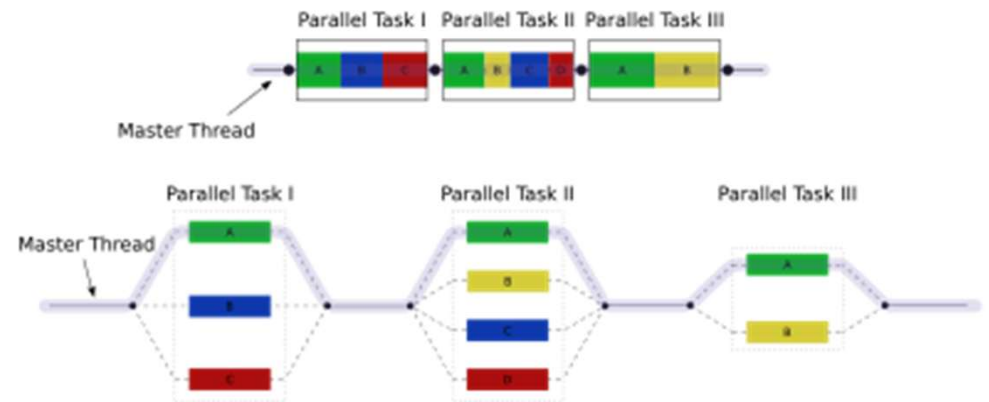
Task parallel programming

- Create tasks to be executed in parallel
- Don't mention threads
threads will internally run the tasks to completion
- Specify task dependencies for tasks to wait
Mention synchronization points among tasks

Fork Join Parallelism

- Fork = create tasks
- Join = task synchronization

Synchronization = sync



Fork Join

- Fork => Spawn tasks for parallel execution
- Join => Sync key word

Fibonacci algorithm

- i th Fibonacci number F_i

$$F_i = \begin{cases} 0 & \text{if } i = 0, \\ 1 & \text{if } i = 1, \\ F_{i-1} + F_{i-2} & \text{if } i \geq 2. \end{cases}$$

Fibonacci algorithm with recurrence relation

```
FIB( $n$ )  
1  if  $n \leq 1$   
2      return  $n$   
3  else  $x = \text{FIB}(n - 1)$   
4       $y = \text{FIB}(n - 2)$   
5      return  $x + y$ 
```

To analyze this algorithm, let $T(n)$ denote the running time of $\text{FIB}(n)$. Since $\text{FIB}(n)$ contains two recursive calls plus a constant amount of extra work, we obtain the recurrence

$$T(n) = T(n - 1) + T(n - 2) + \Theta(1) .$$

Tree of recursive calls for Fib(6)

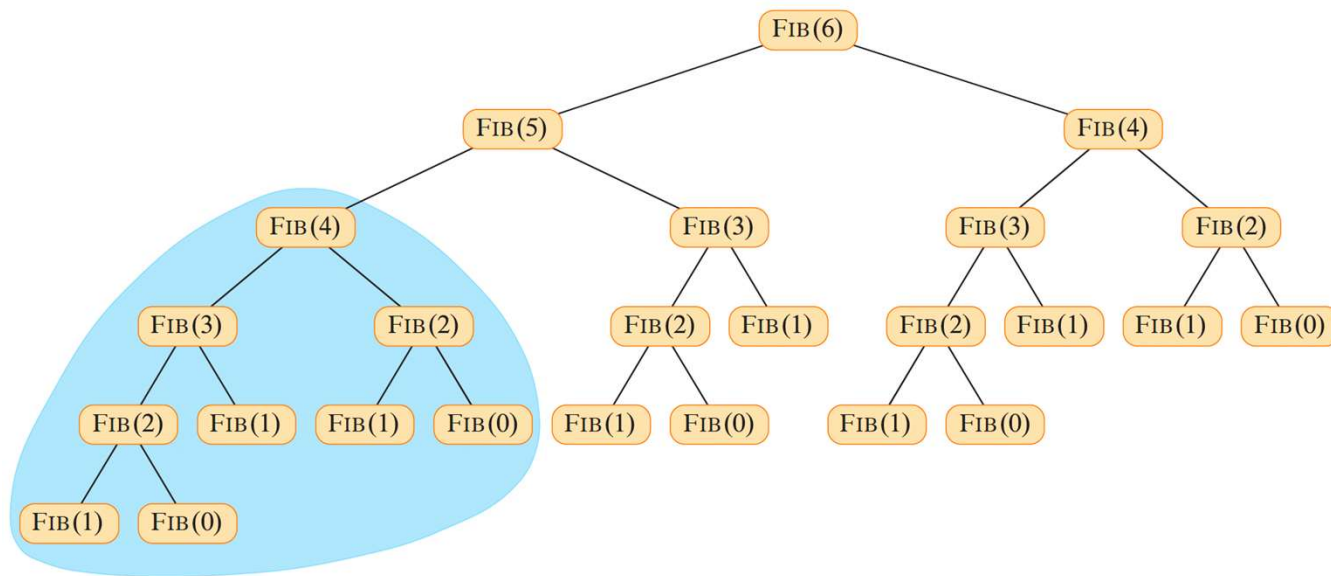


Figure 26.1 The invocation tree for FIB(6). Each node in the tree represents a procedure instance whose children are the procedure instances it calls during its execution. Since each instance of FIB with the same argument does the same work to produce the same result, the inefficiency of this algorithm for computing the Fibonacci numbers can be seen by the vast number of repeated calls to compute the same thing. The portion of the tree shaded blue appears in task-parallel form in Figure 26.2.

Parallel Fibonacci

P-FIB(n)

```
1  if  $n \leq 1$ 
2      return  $n$ 
3  else  $x = \text{spawn P-FIB}(n - 1)$     // don't wait for subroutine to return
4       $y = \text{P-FIB}(n - 2)$           // in parallel with spawned subroutine
5      sync                        // wait for spawned subroutine to finish
6      return  $x + y$ 
```

Parallel Fibonacci

- Express Logical Parallelism

```
P-FIB( $n$ )  
1  if  $n \leq 1$   
2      return  $n$   
3  else  $x = \text{spawn}$  P-FIB( $n - 1$ )    // don't wait for subroutine to return  
4       $y = \text{P-FIB}(n - 2)$            // in parallel with spawned subroutine  
5      sync                         // wait for spawned subroutine to finish  
6      return  $x + y$ 
```

- Sync: Children finish before their parent finishes
like join method in multi-threaded code

Write a task parallel breakBundleOfSticks Algorithm

```
void breakBundleOfSticks( int bundle[], int first, int last )
{
    if ( first == last)
        break a stick bundle[ first ];
    else {
        mid = find index of middle stick
        breakBundleOfSticks ( bundle, first, mid )
        breakBundleOfSticks ( bundle, mid+1, last )
    }
```

Write a Task Parallel findMin Algorithm

```
int findMin( int arr[], int first, int last )
{
    if ( first == last)
        return arr[first];
    else {
        mid = find middle index
        int lMin = spawn findMin( arr, first, mid)
        int rMin = findMin( arr, mid+1, last )
        sync
        return min( lMin, rMin)
    }
}
```

A graph model for parallel execution

- Parallel trace

OR

Computation directed acyclic graph

$G(V, E)$

vertex represent executed instructions.

edge (u, v) represent dependency between instructions u and v .

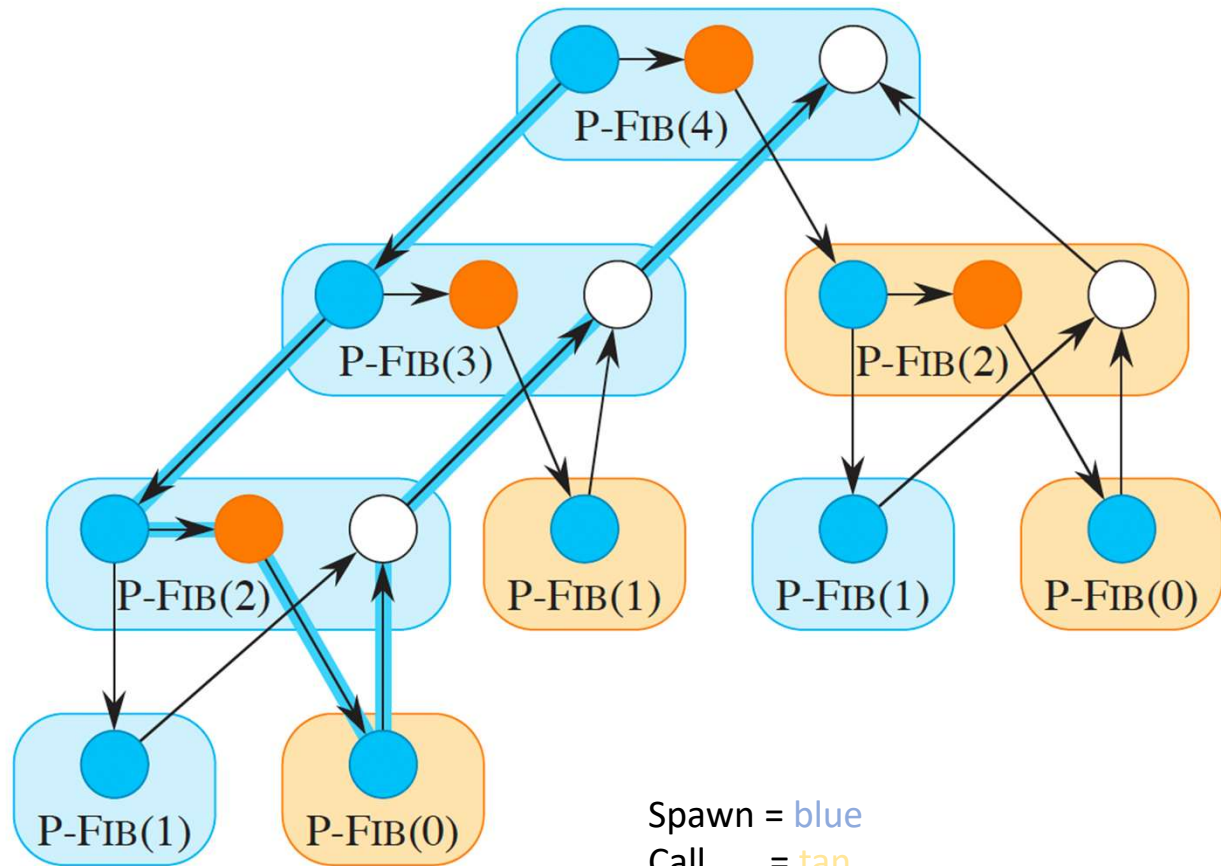
edge $(u, v) \Rightarrow$ instruction u should execute before v .

Parallel Fibonacci

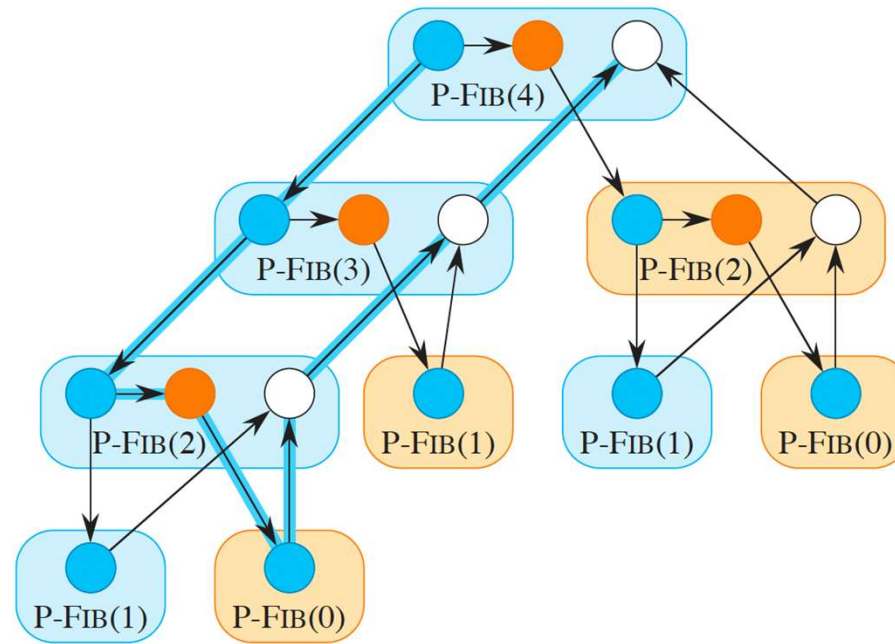
Strand = group of instructions
except spawn, sync, call, return

P-FIB(n)

```
1  if  $n \leq 1$   
2    return  $n$   
3  else  $x = \text{spawn P-FIB}(n - 1)$   
4       $y = \text{P-FIB}(n - 2)$   
5    sync  
6    return  $x + y$ 
```



Work = 17 (17 strands)
Span = 8 (8 strands)



```

P-FIB( $n$ )
1  if  $n \leq 1$ 
2      return  $n$ 
3  else  $x = \text{spawn P-FIB}(n - 1)$ 
4       $y = \text{P-FIB}(n - 2)$ 
5      sync
6      return  $x + y$ 

```

Figure 26.2 The trace of P-FIB(4) corresponding to the shaded portion of Figure 26.1. Each circle represents one strand, with blue circles representing any instructions executed in the part of the procedure (instance) up to the spawn of P-FIB($n - 1$) in line 3; orange circles representing the instructions executed in the part of the procedure that calls P-FIB($n - 2$) in line 4 up to the **sync** in line 5, where it suspends until the spawn of P-FIB($n - 1$) returns; and white circles representing the instructions executed in the part of the procedure after the **sync**, where it sums x and y , up to the point where it returns the result. Strands belonging to the same procedure are grouped into a rounded rectangle, blue for spawned procedures and tan for called procedures. Assuming that each strand takes unit time, the work is 17 time units, since there are 17 strands, and the span is 8 time units, since the critical path—shown with blue edges—contains 8 strands.

Work and Span law

- Work = overall amount of computational work
= sequential time complexity
- Span = critical path length
= height of dependency tree

Work Law

- T_p = parallel computation time
- P = number of processors
- T_1 = time by 1 processor = work

In one step, an ideal parallel computer with P processors can do at most P units of work, and thus in T_P time, it can perform at most $P T_P$ work. Since the total work to do is T_1 , we have $P T_P \geq T_1$. Dividing by P yields the *work law*:

$$T_P \geq T_1/P. \tag{26.2}$$

Work law

$$PT_p \geq T_1$$

$$P \geq T_1/T_p$$

$$T_1/T_p \leq P \quad (\text{speedup can be at most } P)$$

$$\text{Speedup} = T_1/T_p$$

Span Law

T_{∞} = Parallel time with unlimited number of processors.

$$T_P \geq T_{\infty} .$$

A P-processor ideal parallel computer cannot run any faster than a machine with an unlimited number of processors.

Parallelism

- Parallelism = $T_1(n)/T_\infty(n)$

Speedup = sequential time/Parallel time = T_1/T_p

When $T_1/T_p = O(P)$, we say linear speedup

Parallelism

- Parallelism = $T_1(n)/T_\infty(n)$

Speedup = Sequential time/Parallel time

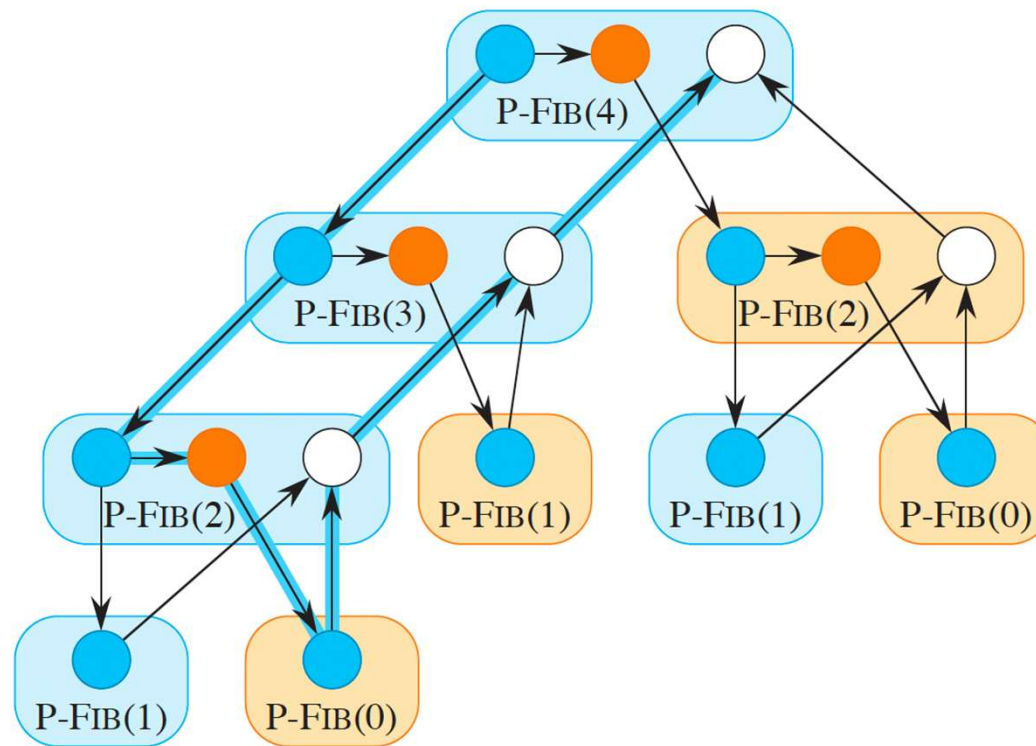
- 1) Parallelism as a ratio: Average amount of work performed in parallel for each step along the critical path.
- 2) Parallelism as Upper bound: Max Speedup that can be achieved on any number of processors.

Parallelism with respect to P

To get good linear speedup, number of processors employed should be less than the parallelism in the algorithm

$$P < T_1 / T_\infty$$

Find parallelism of Fibonacci trace

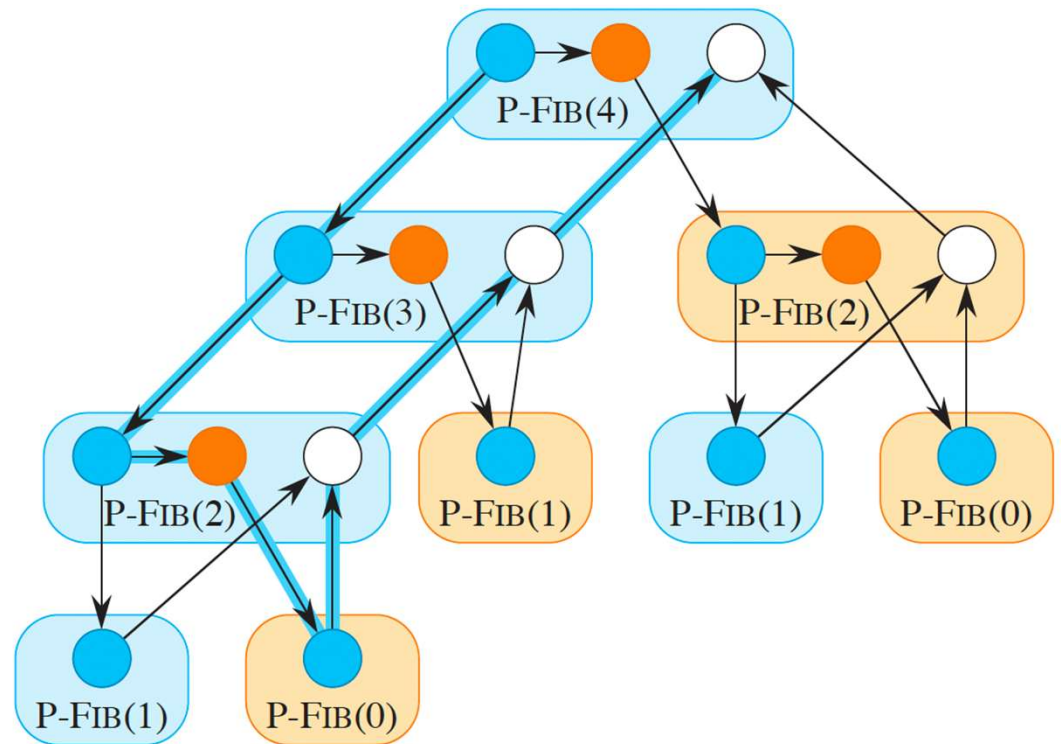


Find parallelism of Fibonacci trace

Parallelism = $17/8 = 2.12$

Achieving high performance
is not possible by increasing
the processor count

For larger input, substantial
performance possible



Scheduling tasks

- Static scheduling of tasks on two processors
Seven tasks t1 to t7 with different execution times

t1 takes one sec,

t2 takes two sec,

t3 takes three sec,

t4 takes four sec,

t5 takes five seconds,

t6 takes five seconds,

t7 takes ten seconds

Calculate speedup on two processors?

Scheduling tasks

- **Dynamic** scheduling of tasks on two processors
Seven tasks t1 to t7 with different execution times

t1 takes one sec,

t2 takes two sec,

t3 takes three sec,

t4 takes four sec,

t5 takes five seconds,

t6 takes five seconds,

t7 takes ten seconds

Calculate speedup on two processors?

Optimal Scheduling of 7 tasks

Show the distribution of tasks among 2 processors that will give the greatest speedup.

In other words, to maximize speedup which thread should handle a given task among all tasks.

t1 takes one sec,

t2 takes two sec,

t3 takes three sec,

t4 takes four sec,

t5 takes five seconds,

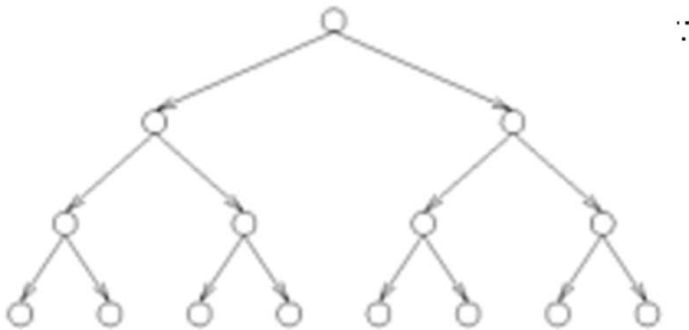
t6 takes five seconds,

t7 takes ten seconds

Speedup in a task dependency graph

Critical path length

Maximum speedup possible?



Critical path length

- Maximum Speedup

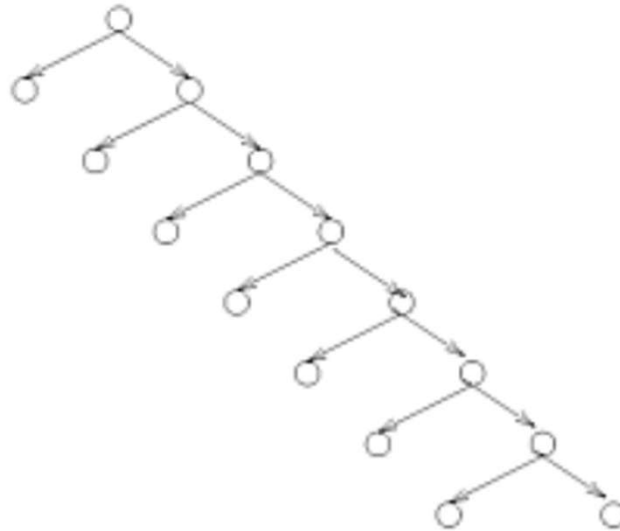
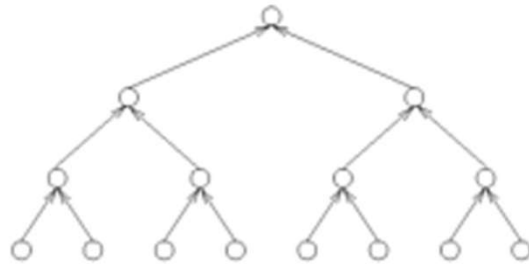
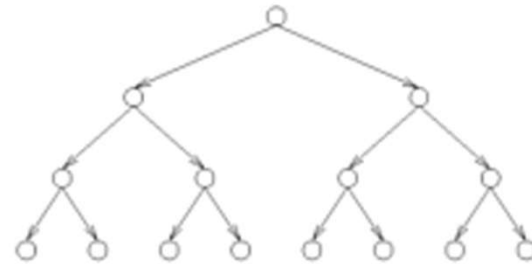


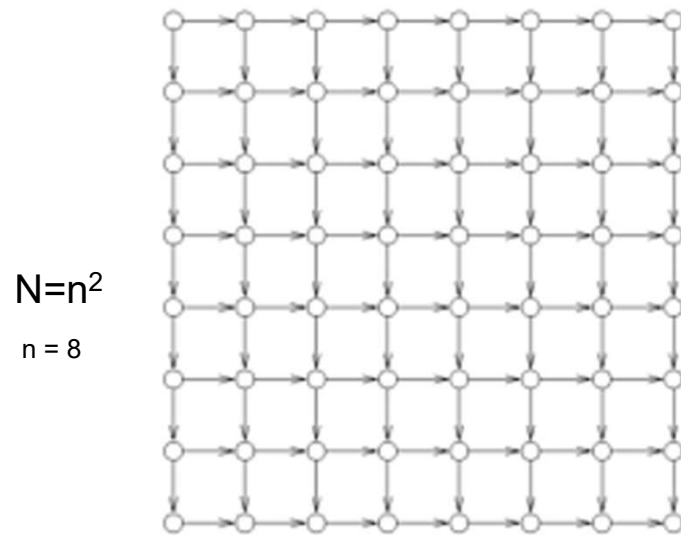
Figure 5.11. Dependency graphs for Problem 5.3.



$N=2^n - 1, n = 4$ (a)

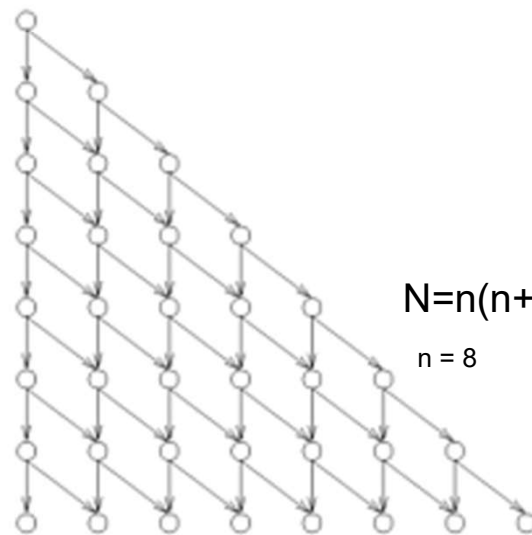


(b) $N=2^n - 1, n = 4$



$N=n^2$
 $n = 8$

(c)



$N=n(n+1)/2$
 $n = 8$

(d)

Scheduling tasks to processors

- Greedy scheduler

Dynamically schedule the tasks to processors so that processor idling is minimized.

Scheduling tasks to processors

- Work law:

Fastest running time T_p on P processors must be at least T_1/P

- Span law:

Fastest possible running time must be at least T_∞

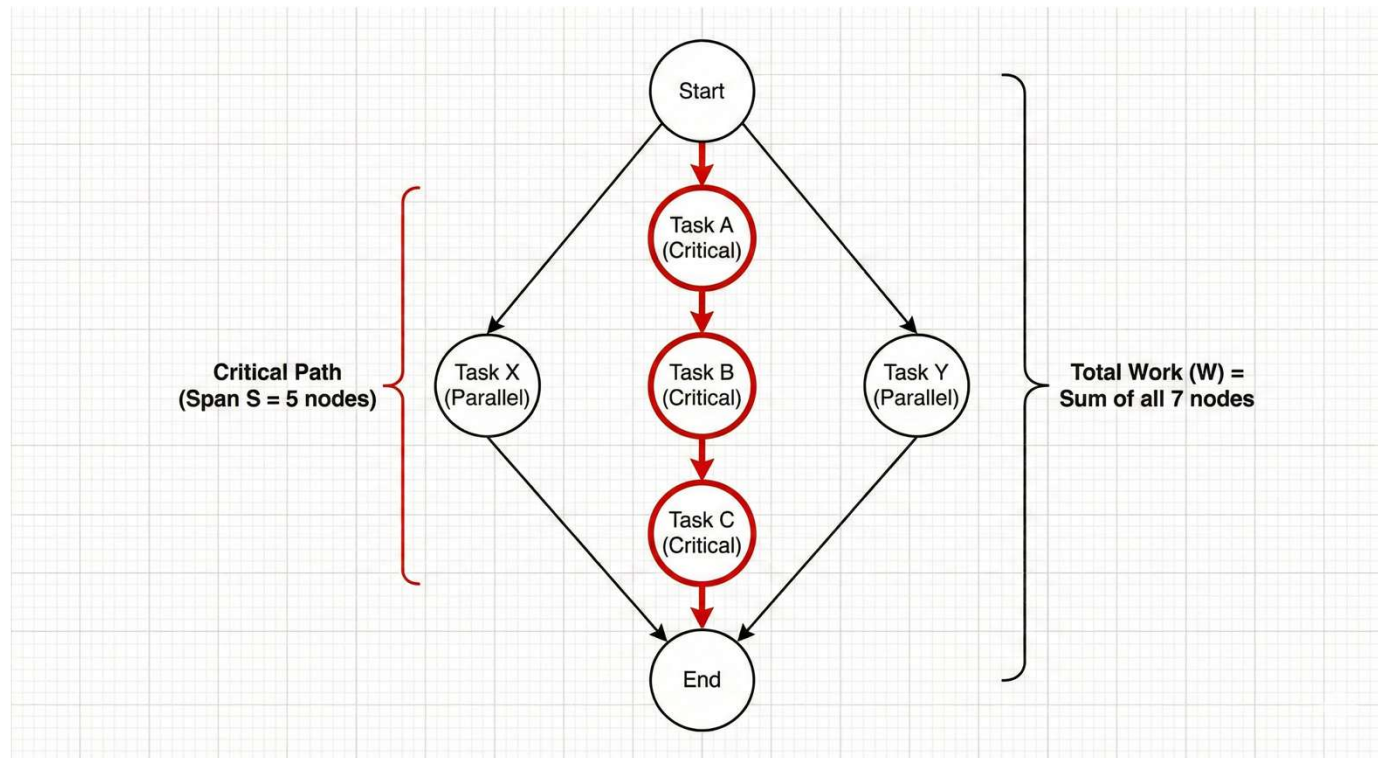
Greedy Scheduling

Theorem 26.1

On an ideal parallel computer with P processors, a greedy scheduler executes a task-parallel computation with work T_1 and span T_∞ in time

$$T_P \leq T_1/P + T_\infty . \tag{26.4}$$

Work and Span in a task dependency graph



Usage of work-span law

- MIT Chess program designed for 512 processors
- But, only 32 processors available for testing
- So, extrapolation necessary

Application of Work Span laws

Unoptimized Chess program	Optimized Chess program
$T_1 = 2048s$	$T_1 = 1024s$
$T_{inf} = 1s$	$T_{inf} = 8s$

Application of Work Span laws

Unoptimized	Optimized
$T_1 = 2048s$	$T_1 = 1024s$
$T_{inf} = 1s$	$T_{inf} = 8s$

Compute the performance using 32 processors and 512 processors

Calculate Parallel time T_p

Application of Work Span laws

UnOptimized chess program	Optimized chess program
$T_1 = 2048s$ $T_{inf} = 1s$	$T_1 = 1024s$ $T_{inf} = 8s$

Using: $T_p = T_1/P + T_{inf}$

P = 512 UnOptimized	P = 512 Optimized	P = 32 UnOptimized	P = 32 Optimized
$T_{512} \leq$	$T_{512} \leq$	$T_{32} \leq 2048/32 + 1$ $= 65 \text{ s}$	$T_{32} \leq 1024/32 + 8$ $= 40 \text{ s}$

Work Law: $T_P \geq T_1/P$.

Theorem 26.1

On an ideal parallel computer with P processors, a greedy scheduler executes a task-parallel computation with work T_1 and span T_∞ in time

$$T_P \leq T_1/P + T_\infty . \quad (26.4)$$

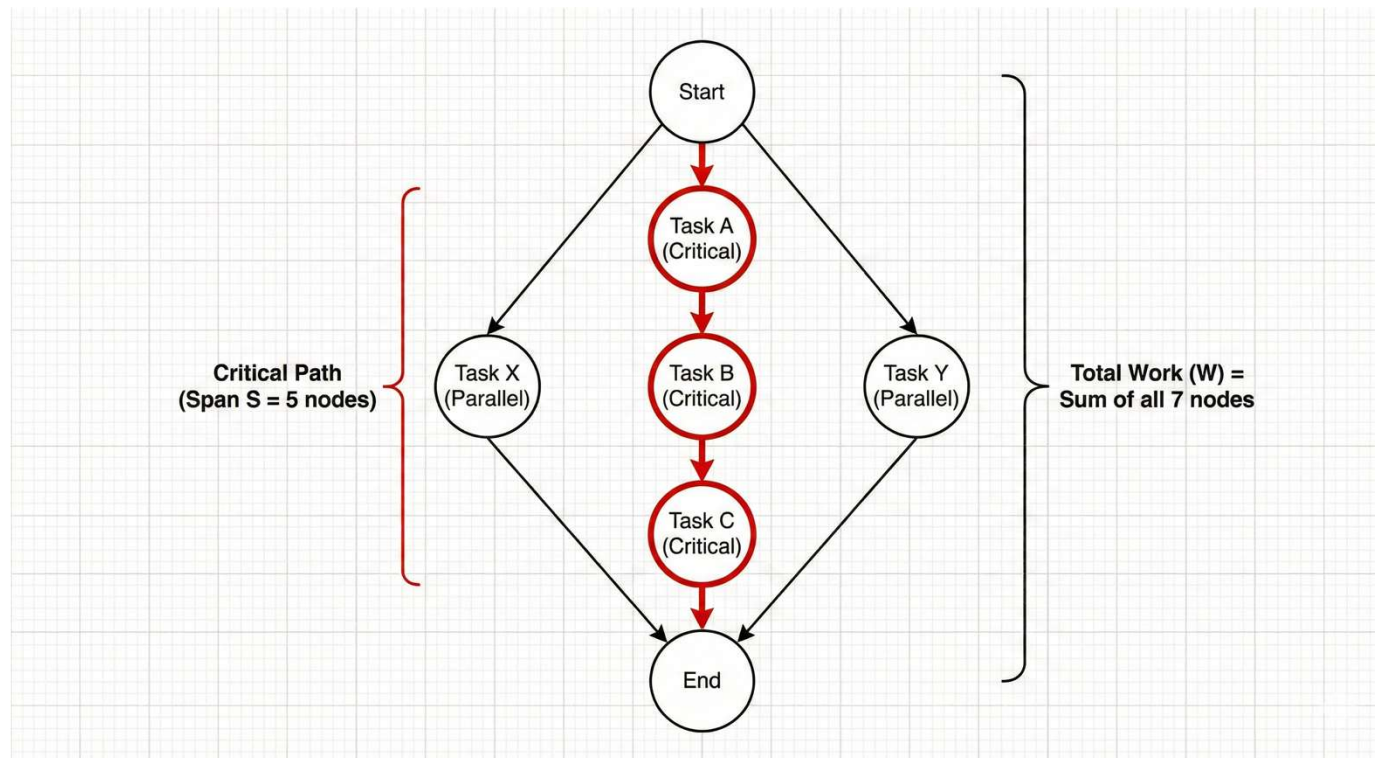
Brent's Law = bound on Parallel time using p processors

$$\frac{T_1}{p} \leq T_p \leq \frac{T_1}{p} + T_\infty .$$

showing that the span (depth) T_∞ and the work T_1 together provide reasonable bounds on the computation time.

If we ran this on **P=2 processors**: Work = 7 Span = 5
The bound states: $T_p \leq (7/2) + 5$. So, $T_p \leq 3.5 + 5$, or $T_p \leq 8.5$

A greedy schedule on 2 processors for this graph would take 5 steps, which satisfies the bound.



Greedy scheduling vs Optimal scheduling

Corollary 26.2

The running time T_P of any task-parallel computation scheduled by a greedy scheduler on a P -processor ideal parallel computer is within a factor of 2 of optimal.

Proof Let T_P^* be the running time produced by an optimal scheduler on a machine with P processors, and let T_1 and T_∞ be the work and span of the computation, respectively. Since the work and span laws—inequalities (26.2) and (26.3)—give $T_P^* \geq \max \{T_1/P, T_\infty\}$, Theorem 26.1 implies that

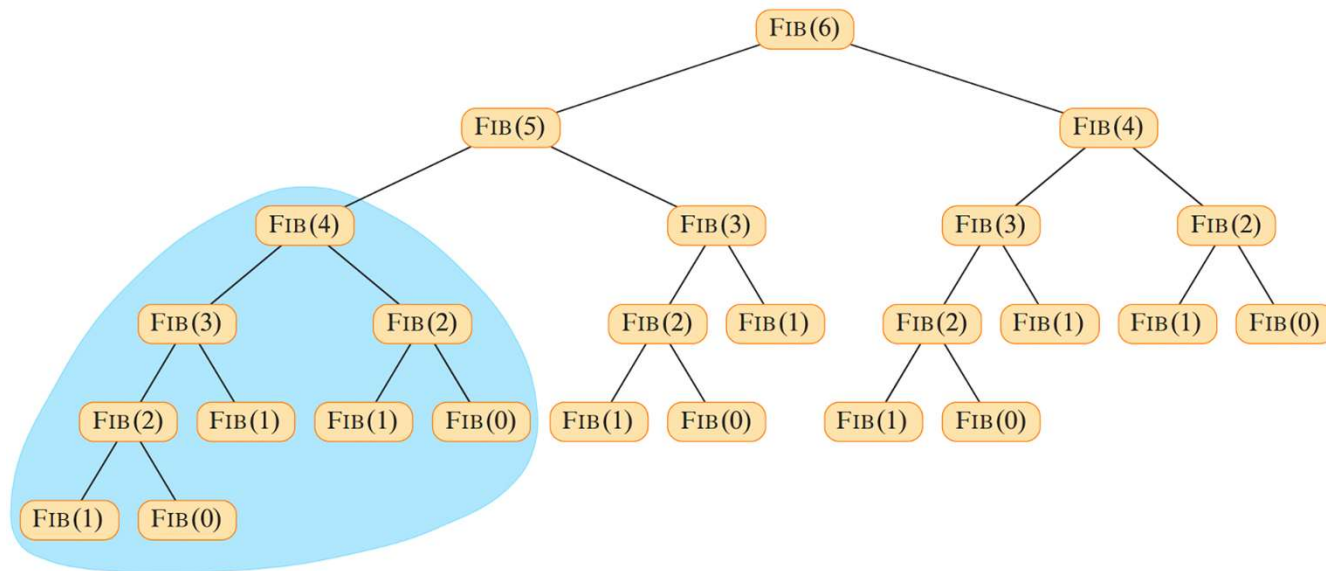
$$\begin{aligned} T_P &\leq T_1/P + T_\infty \\ &\leq 2 \cdot \max \{T_1/P, T_\infty\} \\ &\leq 2T_P^* . \end{aligned}$$

$$\begin{array}{l} 26.2 \\ PT_p \geq T_1 \end{array}$$

$$\begin{array}{l} 26.3 \\ T_P \geq T_\infty . \end{array}$$

Fibonacci

- Parallelism = Work/Span = $O(2^n/n)$



HW Question

- **A fork-join parallel algorithm to do matrix multiplication is reported in a research paper with the following results. Find the inconsistency in these results.**
- 4 processors take 80 seconds.
- 10 processors take 42 seconds.
- 64 processors take 10 seconds.

Hint: Use work and span law along with Brent's theorem to refute the research results:

$$T_P \leq \frac{T_1 - T_\infty}{P} + T_\infty$$

4 processors take 80 seconds.
 10 processors take 42 seconds.
 64 processors take 10 seconds.

	P = 4; T ₄ = 80s	P = 10; T ₁₀ = 42s	P = 64; T ₆₄ = 10s
Work law T _P ≥ T ₁ /P	T ₄ ≥ T ₁ /4 80 ≥ T ₁ /4 T₁ ≤ 320 (tight)	T ₁₀ ≥ T ₁ /10 42 ≥ T ₁ /10 T ₁ ≤ 420	T ₆₄ ≥ T ₁ /64 10 ≥ T ₁ /64 T ₁ ≤ 640
Span law T _{inf} ≤ T _P	T _{inf} ≤ T ₄ T _{inf} ≤ 80	T _{inf} ≤ T ₁₀ T _{inf} ≤ 42	T _{inf} ≤ T ₆₄ T_{inf} ≤ 10 (tight) Inequality 1

Substitute T_1 and T_{inf} from the table above for T_{10} and solve for unknown T_{inf}

$$T_{10} \leq (T_1 - T_{inf})/10 + T_{inf}$$

$$42 \leq (320 - T_{inf})/10 + T_{inf}$$

$$42 \leq 32 - T_{inf}/10 + T_{inf}$$

$$42 \leq 32 + 9T_{inf}/10$$

Subtracting 32 on both sides,

$$10 \leq 9T_{inf}/10$$

$$9T_{inf}/10 \geq 10$$

$$T_{inf} \geq 100/9$$

$$T_{inf} \geq 11.11 \text{ sec}$$

Earlier we derived in **Inequality 1**:

$$T_{inf} \leq 10 \text{ sec}$$

This is a contradiction on the value of T_{inf}